K. Background Processing & Resource Lifecycle

Standalone capability brief · Suggested module: **Execution Runtime**

Purpose

Standardize the lifecycle, scheduling, resource limits, retry behavior, and observability of background jobs, workers, batch tasks, and streaming consumers.

1. Scope and capability areas

Worker lifecycle

Provide startup readiness, graceful shutdown, draining, and safe interruption handling.

Queue abstraction

Standardize message consumption, acknowledgments, visibility timeouts, and consumer lifecycle.

Scheduling and priority

Support scheduled tasks, priority classes, fairness, and tenant-level concurrency controls.

Retries and dead letters

Define retry classification, backoff, poison-message handling, and dead-letter replay.

Cancellation and deadlines

Allow operators and callers to cancel work and propagate execution budgets.

Isolation and resource limits

Set CPU, memory, concurrency, and execution-time limits per workload class.

Progress and results

Expose job status, progress, result retrieval, and operational history.

2. Shared contracts and standards

- Every job defines an idempotency strategy, timeout, retry policy, resource profile, and ownership metadata.
- Graceful shutdown stops new work and completes or safely releases in-flight work within a bounded period.
- Dead-letter queues have owners, alerting, retention, and replay procedures.
- Priority must not permit permanent starvation of lower-priority work.

3. Recommended implementation approach

- Provide a standard worker SDK and deployment template.
- Separate transient failures from permanent validation or business failures.
- Use visibility timeouts or leases aligned with expected processing duration and heartbeat behavior.
- Instrument queue age, attempt count, execution duration, and terminal outcomes.

- Support safe cancellation checkpoints for long-running jobs.

4. Failure behavior and operational expectations

- Workers should stop accepting new tasks during shutdown and leave uncompleted tasks recoverable.
- Poison messages should be quarantined with diagnostic context, not retried indefinitely.
- Overload should trigger bounded concurrency, backpressure, or admission control.

5. Suggested operational metrics

- Queue depth and oldest-message age
- Job success and retry rates
- Dead-letter volume
- Worker shutdown drain time
- Resource use per workload and tenant

6. Readiness and acceptance checklist

- ■ Workers shut down and restart without losing acknowledged work.
- ■ Retries are bounded and idempotent.
- ■ Operators can inspect, cancel, replay, and recover jobs safely.
- ■ Resource and concurrency limits prevent one tenant from monopolizing the runtime.

Ownership note: assign a named team, service owner, support route, versioning policy, and deprecation process before treating this capability as a platform contract.